Graph and its representations

Last Updated : 05 Oct, 2024

A **Graph** is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The graph is denoted by **G(V, E)**.

Representations of Graph

Here are the two most common ways to represent a graph : For simplicity, we are going to consider only unweighted graphs in this post.

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix Representation

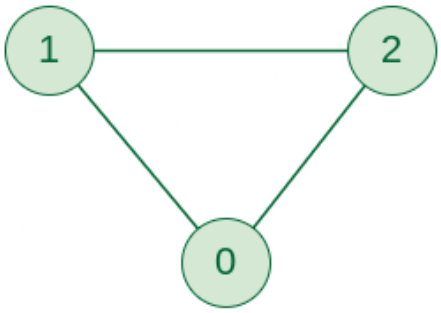
An adjacency matrix is a way of representing a graph as a matrix of boolean (0's and 1's)

Let's assume there are **n** vertices in the graph So, create a 2D matrix **adjMat[n][n]** having dimension n x n.

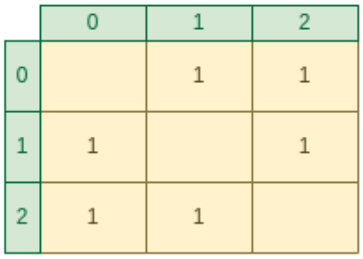
- *If there is an edge from vertex **i** to **j**, mark **adjMat[i][j]** as **1**.*
- *If there is no edge from vertex **i** to **j**, mark **adjMat[i][j]** as **0**.*

Representation of Undirected Graph as Adjacency Matrix:

The below figure shows an undirected graph. Initially, the entire Matrix is initialized to **0**. If there is an edge from source to destination, we insert **1** to both cases (**adjMat[source][destination]** and **adjMat[destination][source])** because we can go either way.



Undirected Graph



Adjacency Matrix

Graph Representation of Undirected graph to Adjacency Matrix

Undirected Graph to Adjacency Matrix

C++ C Java Python C# JavaScript

```cpp
// C++ program to demonstrate Adjacency Matrix
// representation of undirected and unweighted graph
#include <bits/stdc++.h>
using namespace std;

void addEdge(vector<vector<int>> &mat, int i, int j)
{
    mat[i][j] = 1;
    mat[j][i] = 1; // Since the graph is undirected
}

void displayMatrix(vector<vector<int>> &mat)
{
    int V = mat.size();
    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
            cout << mat[i][j] << " ";
        cout << endl;
    }
}

int main()
{

    // Create a graph with 4 vertices and no edges
```

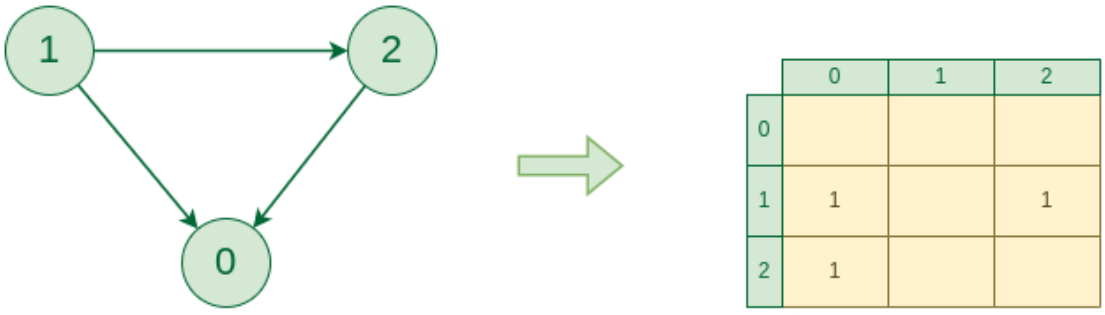
```
    // Note that all values are initialized as 0
    int V = 4;
    vector<vector<int>> mat(V, vector<int>(V, 0));

    // Now add edges one by one
    addEdge(mat, 0, 1);
    addEdge(mat, 0, 2);
    addEdge(mat, 1, 2);
    addEdge(mat, 2, 3);

    /* Alternatively we can also create using below
       code if we know all edges in advacem

     vector<vector<int>> mat = {{ 0, 1, 0, 0 },
                                { 1, 0, 1, 0 },
                                { 0, 1, 0, 1 },
                                { 0, 0, 1, 0 } }; */

    cout << "Adjacency Matrix Representation" << endl;
    displayMatrix(mat);

    return 0;
}
```

Output

```
Adjacency Matrix Representation
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

Representation of Directed Graph as Adjacency Matrix:

The below figure shows a directed graph. Initially, the entire Matrix is initialized to **0**. If there is an edge from source to destination, we insert **1** for that particular **adjMat[source][destination]**.



Directed Graph

Adjacency Matrix

Graph Representation of Directed graph to Adjacency Matrix

Directed Graph to Adjacency Matrix

Adjacency List Representation

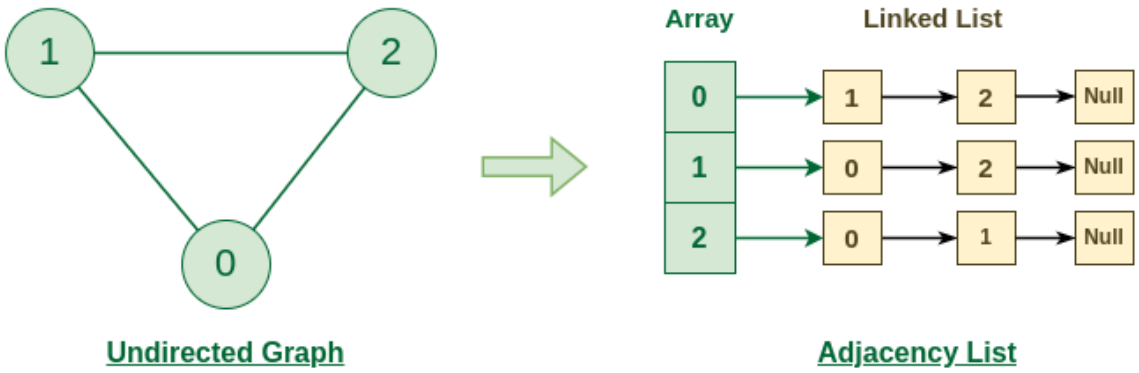
An array of Lists is used to store edges between two vertices. The size of array is equal to the number of **vertices (i.e, n)**. Each index in this array represents a specific vertex in the graph. The entry at the index i of the array contains a linked list containing the vertices that are adjacent to vertex **i**.

Let's assume there are **n** vertices in the graph So, create an **array of list** of size **n** as **adjList[n].**

- *adjList[0] will have all the nodes which are connected (neighbour) to vertex **0**.*
- *adjList[1] will have all the nodes which are connected (neighbour) to vertex **1** and so on.*

Representation of Undirected Graph as Adjacency list:

The below undirected graph has 3 vertices. So, an array of list will be created of size 3, where each indices represent the vertices. Now, vertex 0 has two neighbours (i.e, 1 and 2). So, insert vertex 1 and 2 at indices 0 of array. Similarly, For vertex 1, it has two neighbour (i.e, 2 and 0) So, insert vertices 2 and 0 at indices 1 of array. Similarly, for vertex 2, insert its neighbours in array of list.



Undirected Graph

Adjacency List

Graph Representation of Undirected graph to Adjacency List

Undirected Graph to Adjacency list

C++ C Java Python C# JavaScript

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Function to add an edge between two vertices
void addEdge(vector<vector<int>>& adj, int i, int j) {
    adj[i].push_back(j);
    adj[j].push_back(i); // Undirected
}

// Function to display the adjacency list
void displayAdjList(const vector<vector<int>>& adj) {
    for (int i = 0; i < adj.size(); i++) {
        cout << i << ": "; // Print the vertex
        for (int j : adj[i]) {
            cout << j << " "; // Print its adjacent
        }
        cout << endl;
    }
}

// Main function
int main() {
    // Create a graph with 4 vertices and no edges
    int V = 4;
    vector<vector<int>> adj(V);
```

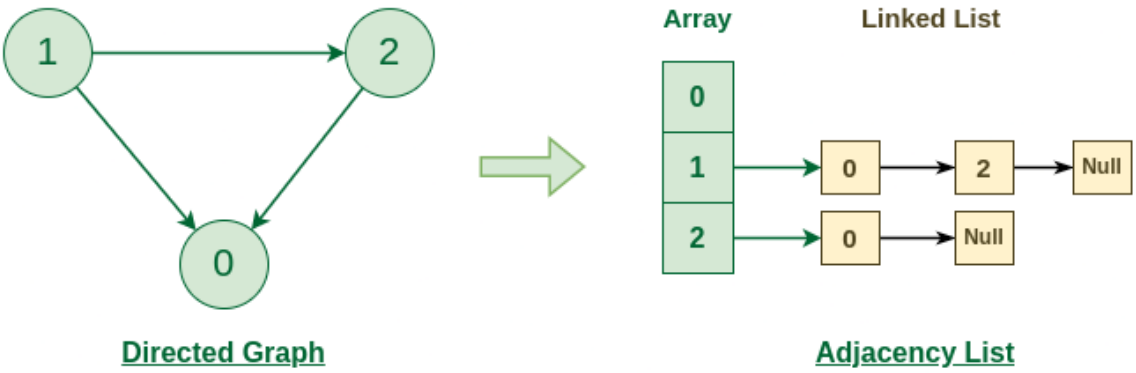
```
    // Now add edges one by one
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
    addEdge(adj, 1, 2);
    addEdge(adj, 2, 3);

    cout << "Adjacency List Representation:" << endl;
    displayAdjList(adj);

    return 0;
}
```

Output

```
Adjacency List Representation:
0: 1 2
1: 0 2
2: 0 1 3
3: 2
```

Representation of Directed Graph as Adjacency list:

The below directed graph has 3 vertices. So, an array of list will be created of size 3, where each indices represent the vertices. Now, vertex 0 has no neighbours. For vertex 1, it has two neighbour (i.e, 0 and 2) So, insert vertices 0 and 2 at indices 1 of array. Similarly, for vertex 2, insert its neighbours in array of list.



Graph Representation of Directed graph to Adjacency List

Directed Graph to Adjacency list

Graph and its representations

Next Article
Types of Graphs with Examples

Similar Reads

Graph Algorithms

Graph algorithms are methods used to manipulate and analyze graphs, solving various range of problems like finding the shortest path, cycles detection. If you are looking for difficulty-wise list of problems,...

3 min read

Introduction to Graph Data Structure

Graph Data Structure is a non-linear data structure consisting of vertices and edges. It is useful in fields such as social network analysis, recommendation systems, and computer networks. In the field of sports...

15+ min read

Graph and its representations

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More...

12 min read

Types of Graphs with Examples

A graph is a mathematical structure that represents relationships between objects by connecting a set of points. It is used to establish a pairwise relationship between elements in a given set. graphs are widely...

9 min read

Basic Properties of a Graph

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph. The basi...

4 min read

Applications, Advantages and Disadvantages of Graph

Graph is a non-linear data structure that contains nodes (vertices) and edges. A graph is a collection of set of vertices and edges (formed by connecting two vertices). A graph is defined as G = {V, E} where V i...

7 min read

Transpose graph

Transpose of a directed graph G is another directed graph on the same set of vertices with all of the edges reversed compared to the orientation of the corresponding edges in G. That is, if G contains an...

9 min read

Difference Between Graph and Tree

Graphs and trees are two fundamental data structures used in computer science to represent relationships between objects. While they share some similarities, they also have distinct differences th...

2 min read

BFS and DFS on Graph

Cycle in a Graph